VoltTwin AI

Battery Health Prediction Using Machine Learning

In [6]:
```
print("Welcome to VoltTwin AI")
```

```
Welcome to VoltTwin AI
```

Data Loading

In [7]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px

print("All libraries imported successfully")
```

```
All libraries imported successfully
```

In [8]:
```
import pandas as pd

df = pd.read_csv("../data/raw/Battery_cycle_level_dataset.csv")
df.head()
```

Out[8]:

	battery_id	cycle	voltage	temperature	capacity	soh	rul
0	B0005	1	3.532781	32.536891	1.861976	1.000000	167
1	B0005	2	3.542968	32.643595	1.851862	0.994568	166
2	B0005	3	3.553056	32.522526	1.840808	0.988631	165
3	B0005	4	3.545849	32.492083	1.850058	0.993599	164
4	B0005	5	3.544456	32.368612	1.849432	0.993263	163

Data Exploration

In [9]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1415 entries, 0 to 1414
Data columns (total 7 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   battery_id   1415 non-null   str
 1   cycle        1415 non-null   int64
 2   voltage      1415 non-null   float64
 3   temperature  1415 non-null   float64
 4   capacity     1415 non-null   float64
 5   soh          1415 non-null   float64
 6   rul          1415 non-null   int64
dtypes: float64(4), int64(2), str(1)
memory usage: 84.4 KB
```

In [10]:
```
df.columns
```

Out[10]:
```
Index(['battery_id', 'cycle', 'voltage', 'temperature', 'capacity', 'soh',
       'rul'],
      dtype='str')
```

In [11]:
```
df
```

Out[11]:

	battery_id	cycle	voltage	temperature	capacity	soh	rul
0	B0005	1	3.532781	32.536891	1.861976	1.000000	167
1	B0005	2	3.542968	32.643595	1.851862	0.994568	166
2	B0005	3	3.553056	32.522526	1.840808	0.988631	165
3	B0005	4	3.545849	32.492083	1.850058	0.993599	164
4	B0005	5	3.544456	32.368612	1.849432	0.993263	163
...	...	...	...	...	...	...	...
1410	B0053	54	3.080080	12.530864	1.292799	0.988320	1
1411	B0053	55	3.063813	12.900294	1.312809	1.003617	0
1412	B0054	1	3.205983	12.489230	0.853855	1.000000	0
1413	B0055	1	3.153346	12.207199	0.854758	1.000000	0
1414	B0056	1	3.241611	10.334364	0.794083	1.000000	0

1415 rows × 7 columns

In [12]:
```
df.describe()
```

Out[12]:

	cycle	voltage	temperature	capacity	soh	rul
count	1415.000000	1415.000000	1415.000000	1415.000000	1415.000000	1415.000000
mean	55.389399	3.420612	26.589773	1.501191	0.849014	54.175972
std	42.728497	0.166460	13.196045	0.316995	0.119219	43.709204
min	1.000000	2.530010	7.684550	0.520105	0.604449	0.000000
25%	21.000000	3.387078	9.555186	1.321640	0.750777	16.500000
50%	43.000000	3.452596	31.139259	1.504115	0.850350	43.000000
75%	87.000000	3.507450	32.753400	1.709022	0.961174	86.000000
max	168.000000	3.753027	54.997372	2.498996	1.048527	167.000000

In [13]:
```
print(df.columns.tolist())
```

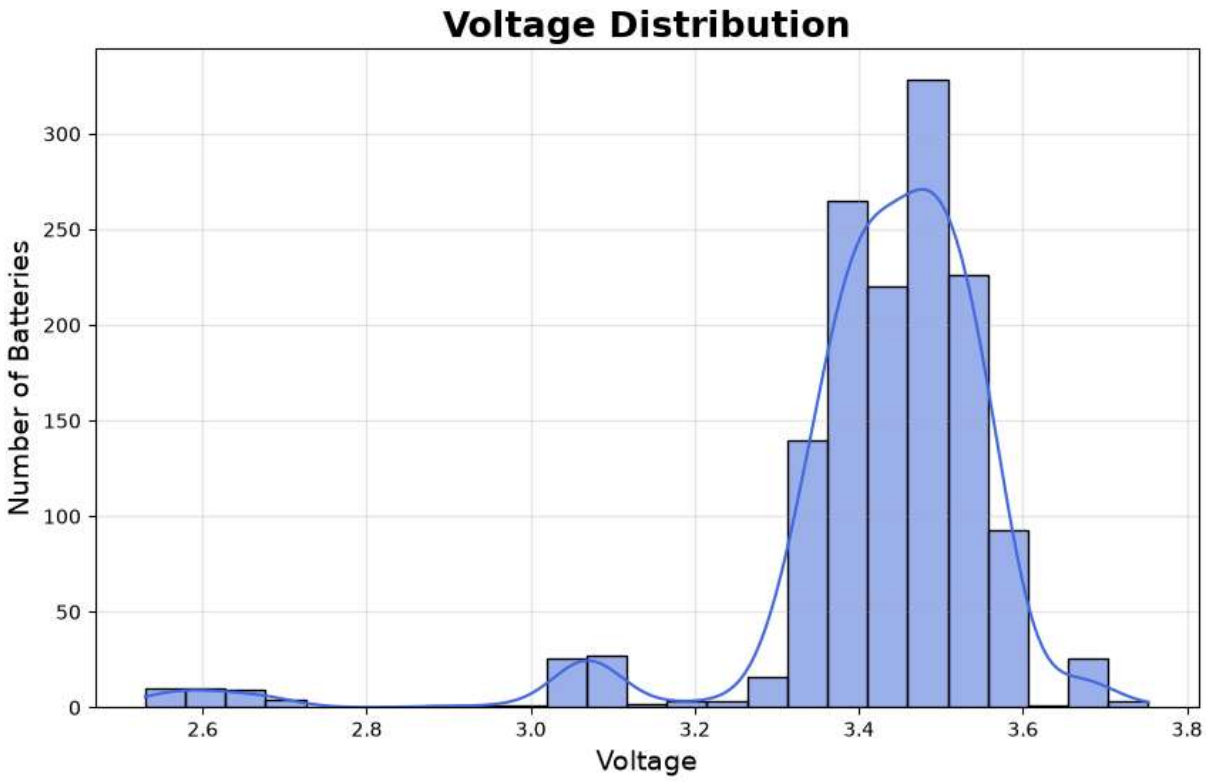
```
['battery_id', 'cycle', 'voltage', 'temperature', 'capacity', 'soh', 'rul']
```

Data Visualization

In [14]:
```
import matplotlib.pyplot as plt
import seaborn as sns

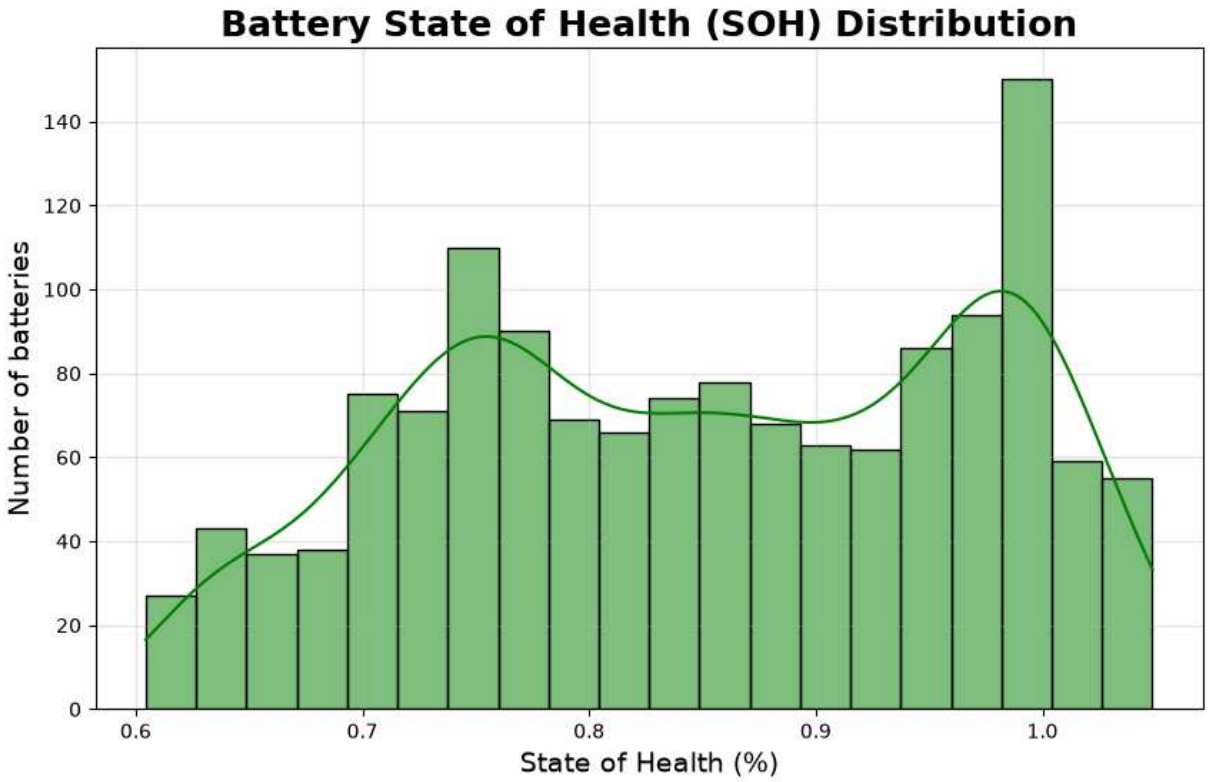
plt.figure(figsize=(10,6))
sns.histplot(
    data=df,
    x="voltage",
    bins=25,
    color="royalblue",
    edgecolor="black",
    kde=True
)

plt.title("Voltage Distribution", fontsize=18, fontweight="bold")
plt.xlabel("Voltage", fontsize=14)
plt.ylabel("Number of Batteries", fontsize=14)
plt.grid(alpha=0.3)
plt.show()
```



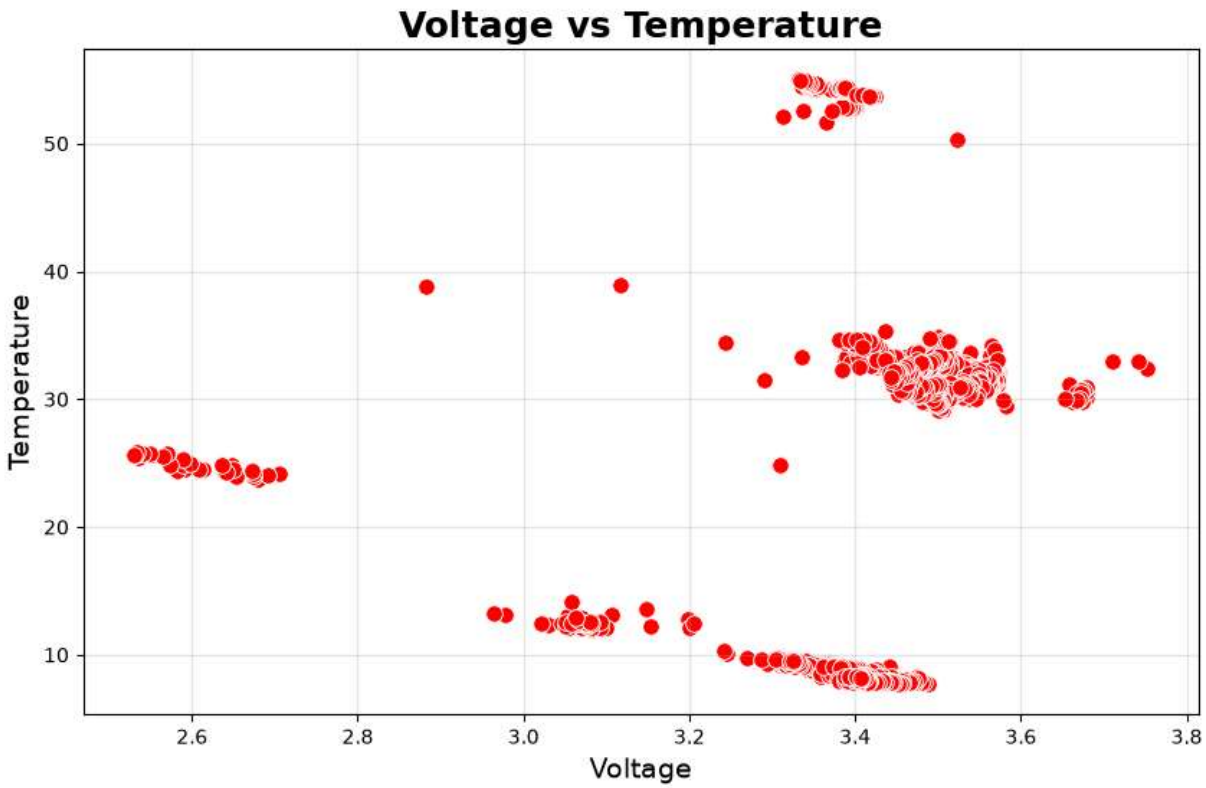
In [15]:
```
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(10,6))
sns.histplot(
    data=df,
    x="soh",
    bins=20,
    color="green",
    edgecolor="black",
    kde=True
)

plt.title("Battery State of Health (SOH) Distribution",
          fontsize=18, fontweight="bold")
plt.xlabel("State of Health (%)", fontsize=14)
plt.ylabel("Number of batteries", fontsize=14)
plt.grid(alpha=0.3)
plt.show()
```



In [16]:
```
plt.figure(figsize=(10,6))

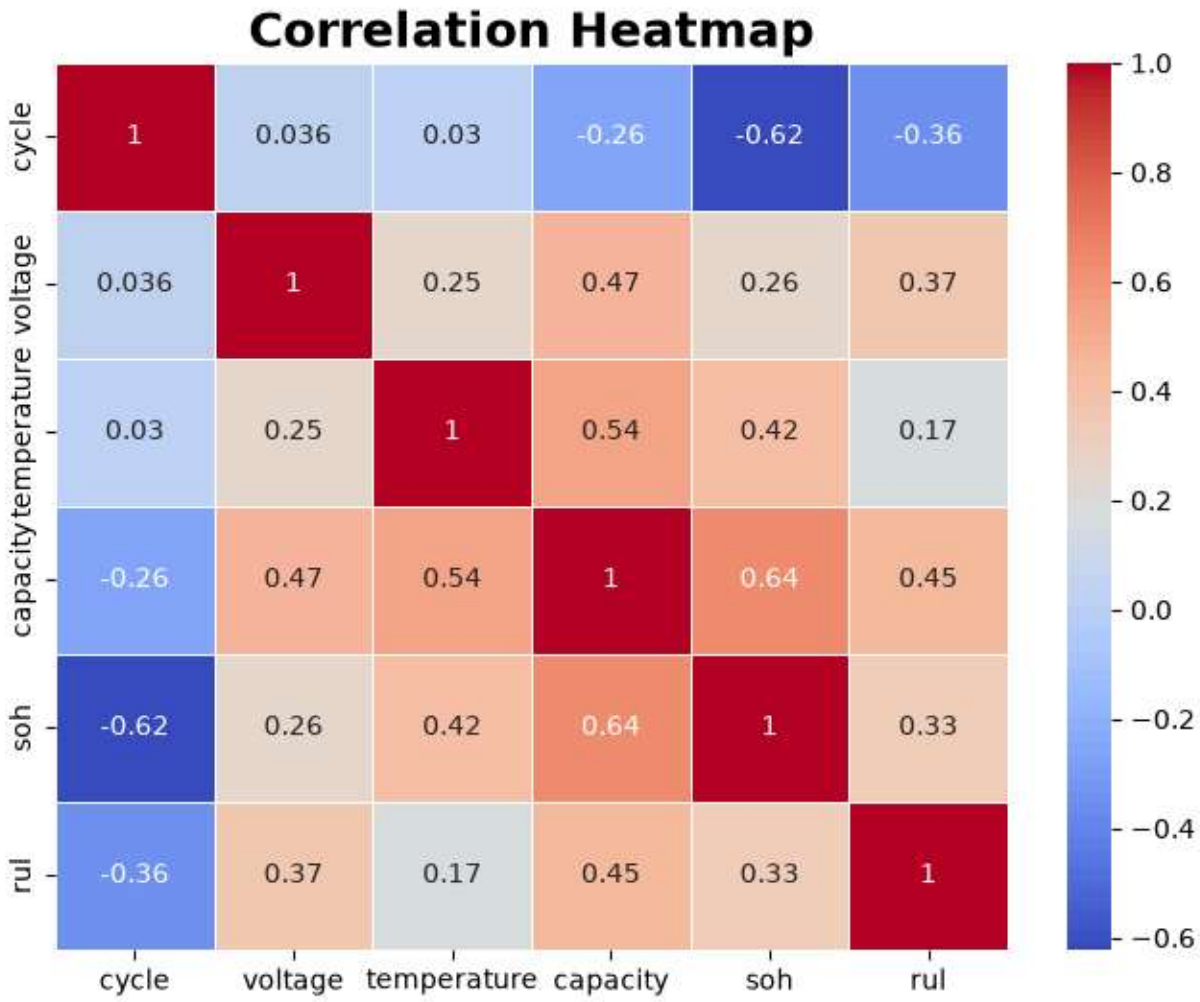
sns.scatterplot(
    data=df,
    x="voltage",
    y="temperature",
    color="red",
    s=70
)
plt.title("Voltage vs Temperature", fontsize=18, fontweight="bold")
plt.xlabel("Voltage", fontsize=14)
plt.ylabel("Temperature", fontsize=14)
plt.grid(alpha=0.3)
plt.show()
```



In [17]:
```
plt.figure(figsize=(8,6))

sns.heatmap(
    df.corr(numeric_only=True),
    annot=True,
    cmap="coolwarm",
    linewidths=0.5
)

plt.title("Correlation Heatmap", fontsize=18, fontweight="bold")
plt.show()
```



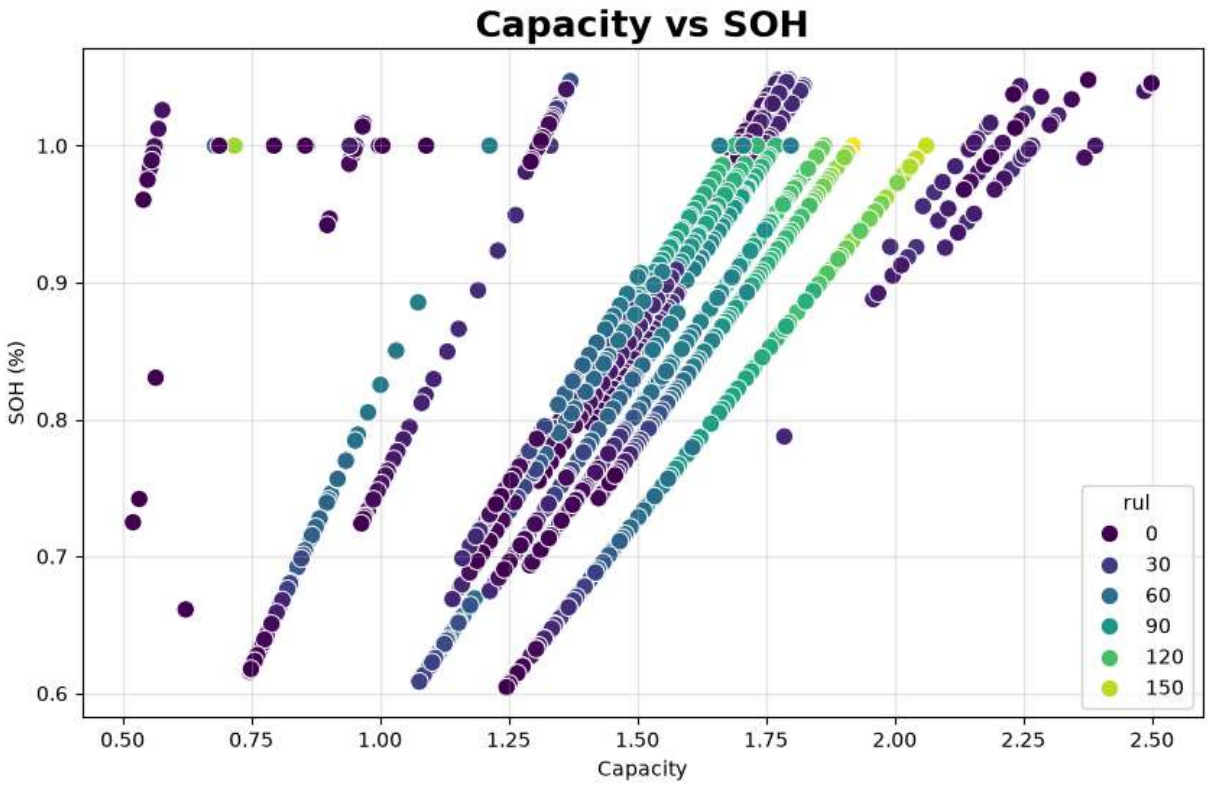
In [18]:
```
plt.figure(figsize=(10,6))

sns.scatterplot(
    data=df,
    x="capacity",
    y="soh",
    hue="rul",
    palette="viridis",
    s=80
)

plt.title("Capacity vs SOH", fontsize=18, fontweight="bold")
plt.xlabel("Capacity")
plt.ylabel("SOH (%)")

plt.grid(alpha=0.3)

plt.show()
```



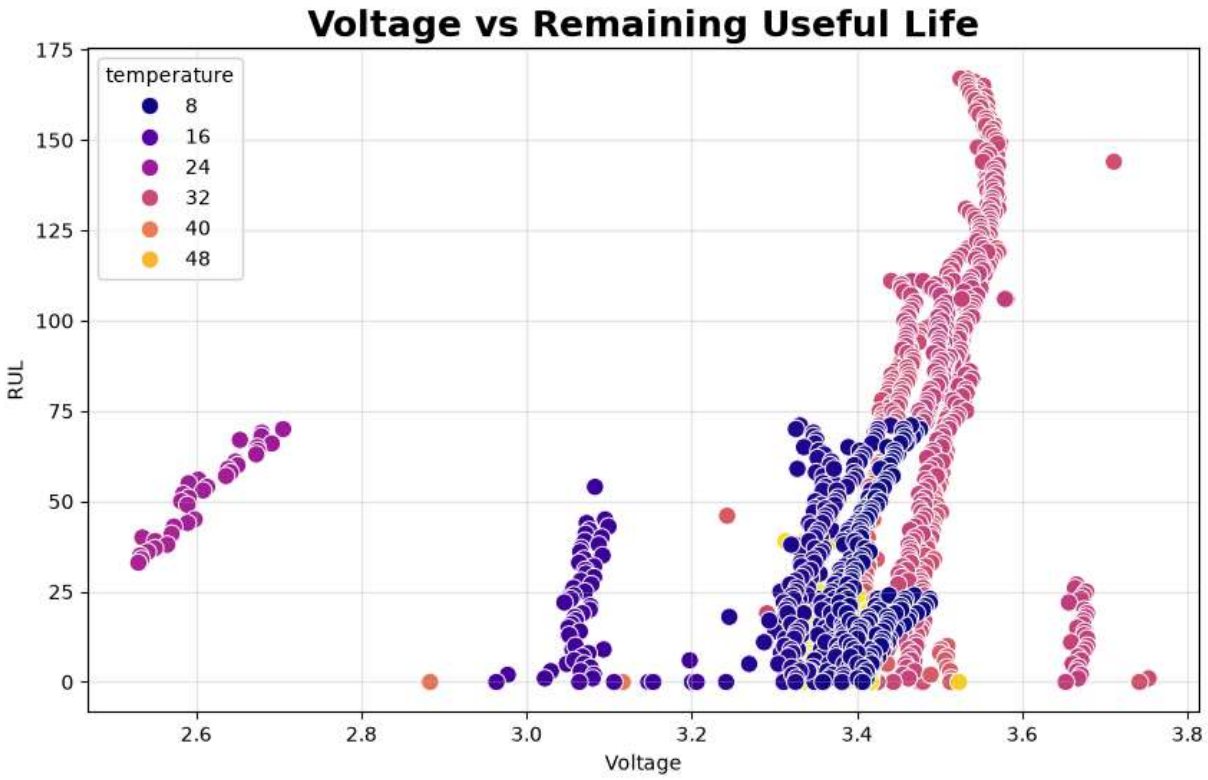
In [19]:
```
plt.figure(figsize=(10,6))

sns.scatterplot(
    data=df,
    x="voltage",
    y="rul",
    hue="temperature",
    palette="plasma",
    s=80
)

plt.title("Voltage vs Remaining Useful Life", fontsize=18, fontweight="bold")
plt.xlabel("Voltage")
plt.ylabel("RUL")

plt.grid(alpha=0.3)

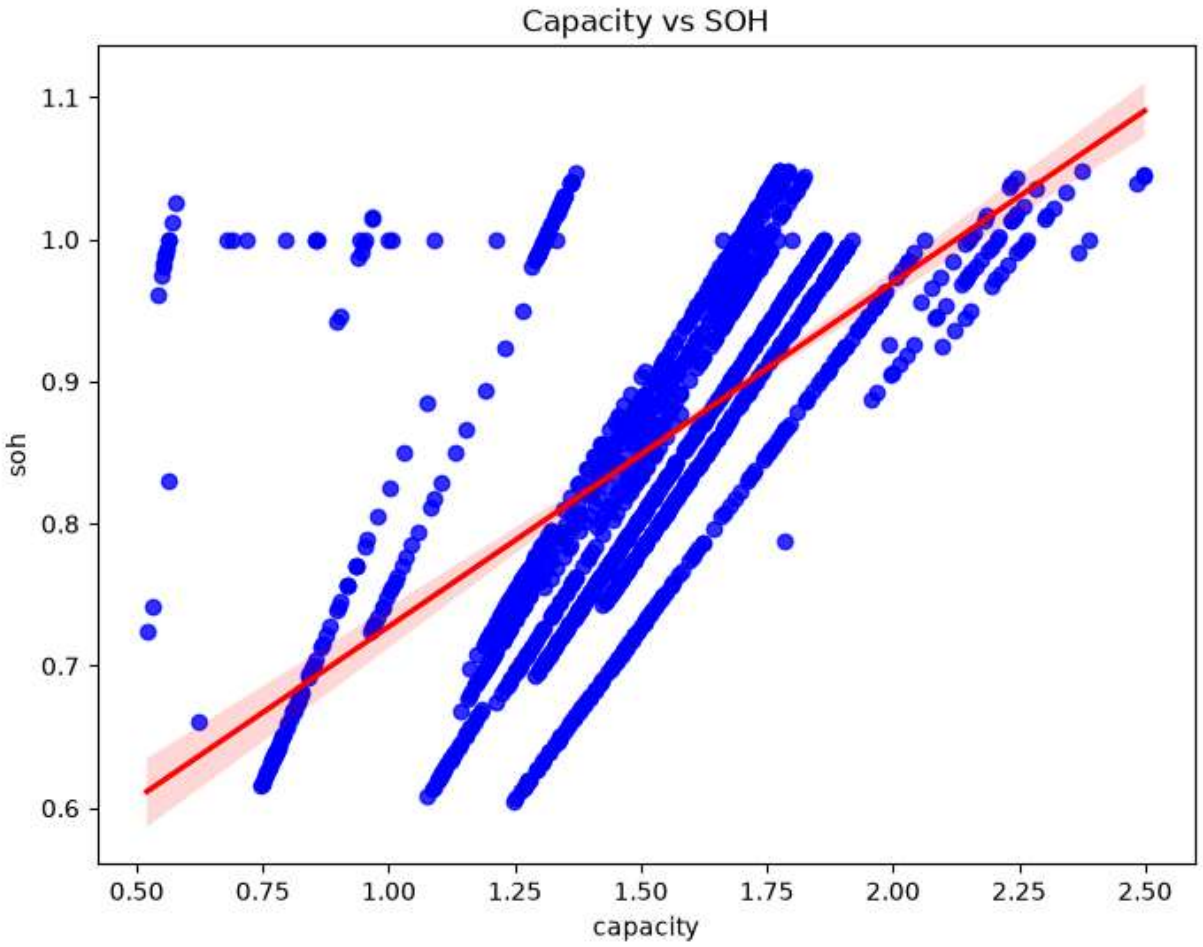
plt.show()
```



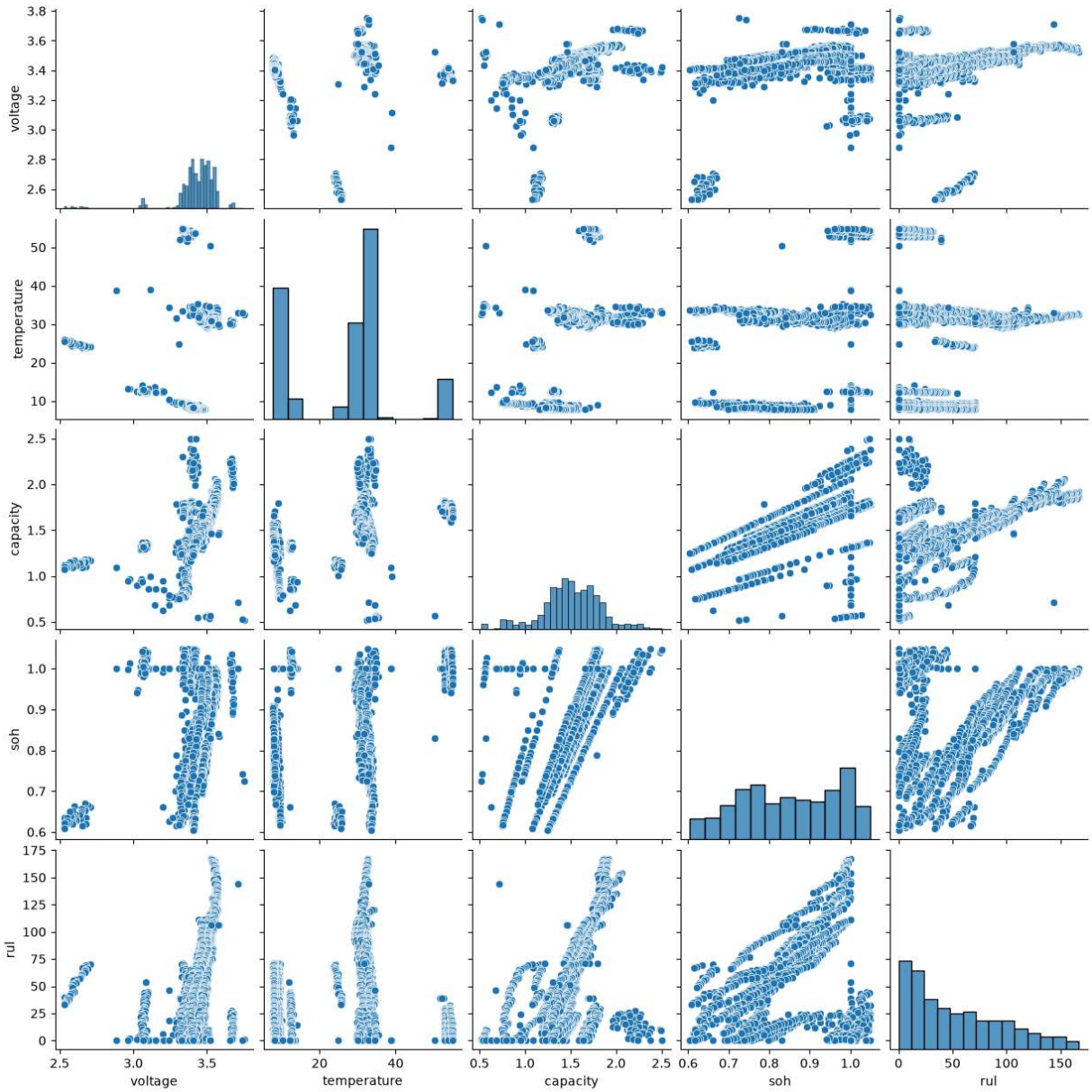
In [20]:
```
plt.figure(figsize=(8,6))

sns.regplot(
    data=df,
    x="capacity",
    y="soh",
    scatter_kws={"color":"blue"},
    line_kws={"color":"red"}
)

plt.title("Capacity vs SOH")
plt.show()
```



In [21]:
```
sns.pairplot(
    df[["voltage","temperature","capacity","soh","rul"]]
)
plt.show()
```

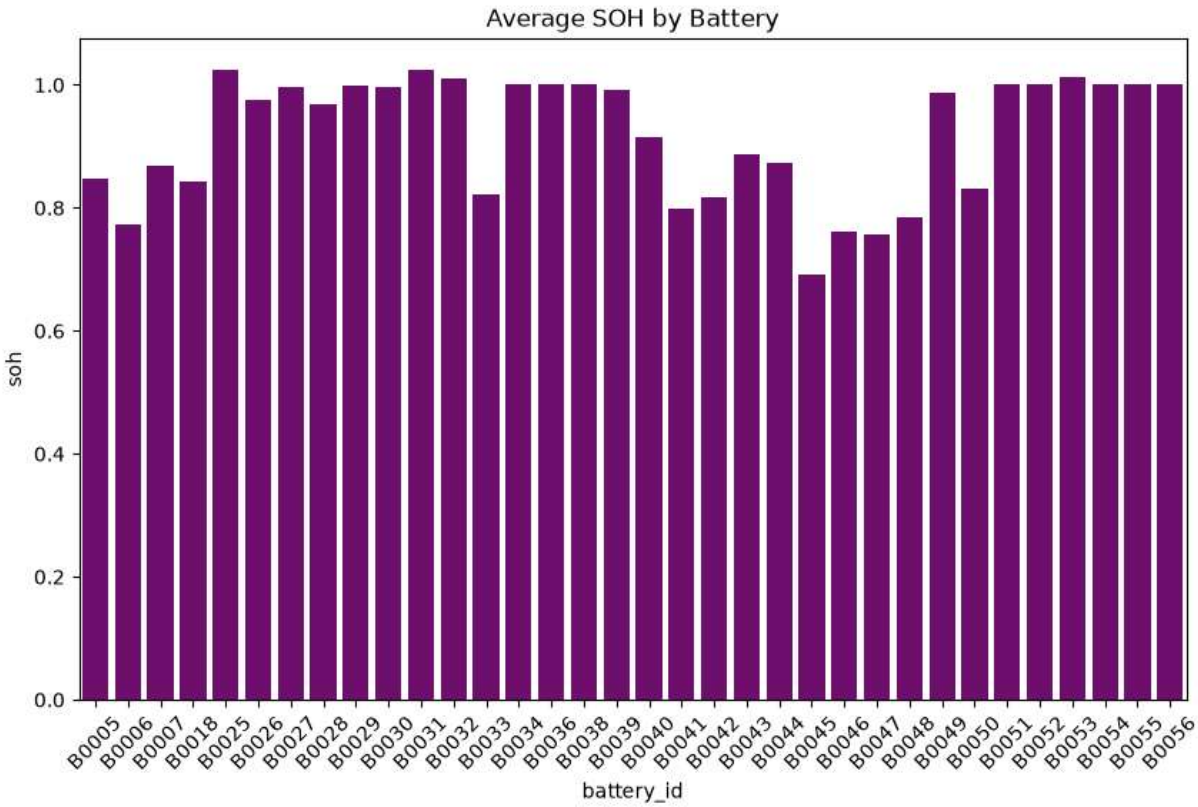


In [22]:
```
plt.figure(figsize=(10,6))

sns.barplot(
    data=df,
    x="battery_id",
    y="soh",
    estimator="mean",
    color="purple",
    errorbar=None
)

plt.title("Average SOH by Battery")
plt.xticks(rotation=45)

plt.show()
```

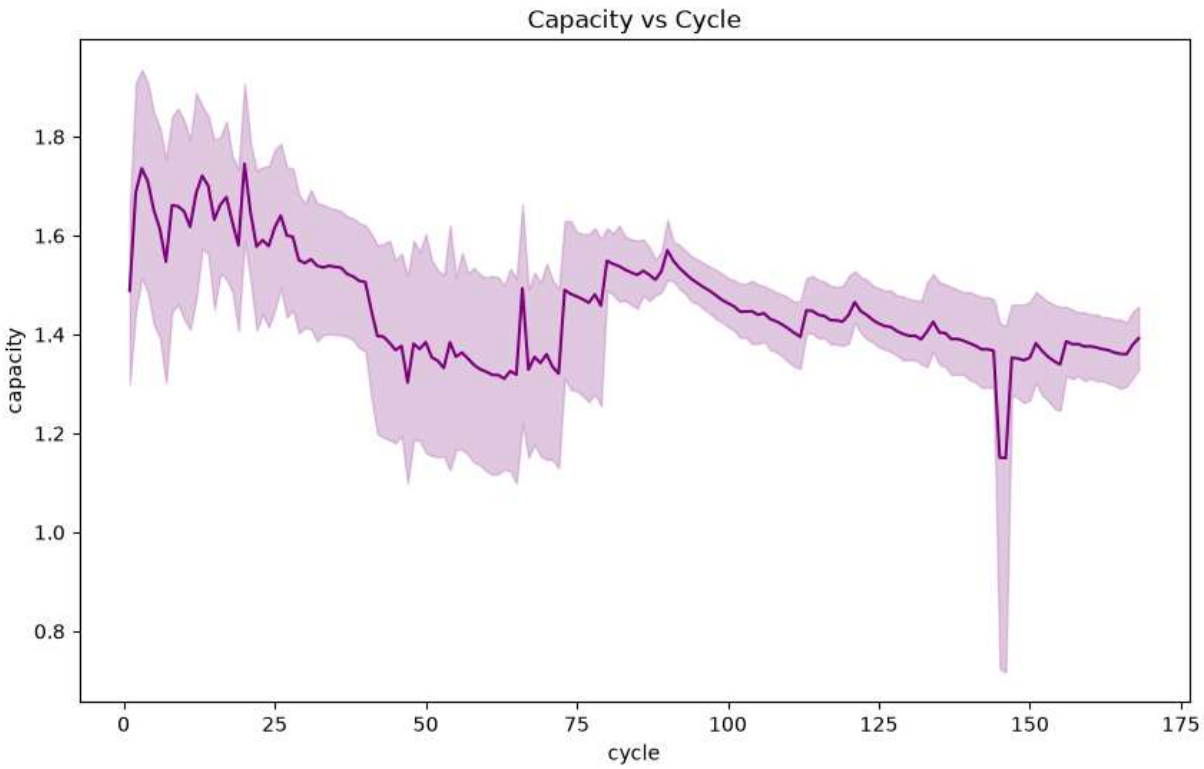


In [23]:
```
plt.figure(figsize=(10,6))

sns.lineplot(
    data=df,
    x="cycle",
    y="capacity",
    color="purple"
)

plt.title("Capacity vs Cycle")

plt.show()
```



Data Preprocessing

In [24]:
```
df.isnull().sum()
```

Out[24]:
```
battery_id     0
cycle          0
voltage        0
temperature    0
capacity       0
soh            0
rul            0
dtype: int64
```

In [25]:
```
df.duplicated().sum()
```

Out[25]:
```
np.int64(0)
```

In [26]:
```
df.shape
```

Out[26]:
```
(1415, 7)
```

In [27]:
```
df.dtypes
```

Out[27]:
```
battery_id         str
cycle            int64
voltage        float64
temperature    float64
capacity       float64
soh            float64
rul              int64
dtype: object
```

Machine Learning

In [28]:
```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error,mean_squared_error, r2_score
```

In [29]:
```
X = df[["cycle","voltage","temperature","capacity","soh"]]
y = df["rul"]
```

In [30]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```

In [31]:
```
model = RandomForestRegressor(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)
```

Out[31]:

▾ RandomForestRegressor

▸ Parameters

▸ Fitted attributes

In [32]:
```
y_pred = model.predict(X_test)
```

Model Evaluation

In [33]:
```
r2 = r2_score(y_test, y_pred)
print("R^2 Score:", r2)
```

```
R^2 Score: 0.9885168099897693
```

In [34]:
```
mea = mean_absolute_error(y_test, y_pred)
print("Mean Absolute Error (MAE):", mea)
```

```
Mean Absolute Error (MAE): 2.3489752650176676
```

In [35]:
```
import numpy as np

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("Root Mean Squared Error (RMSE):", rmse)
```

```
Root Mean Squared Error (RMSE): 4.529601968488276
```

Actual vs Prediction Results

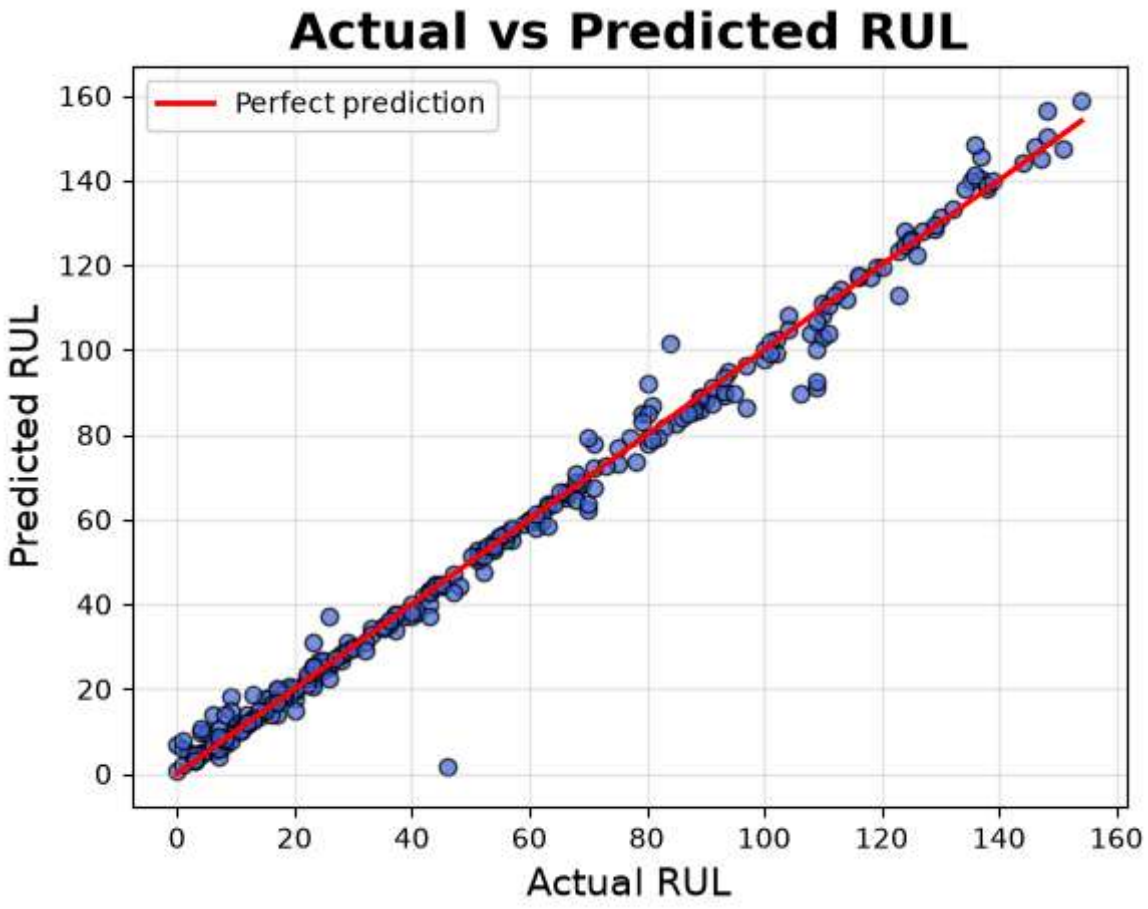
In [36]:
```
plt.scatter(y_test, y_pred,
            color="royalblue",
            alpha=0.7,
            edgecolors="black")

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color="red",
    linewidth=2,
    label="Perfect prediction"
)

plt.title("Actual vs Predicted RUL", fontsize=18, fontweight="bold")
plt.xlabel("Actual RUL", fontsize=14)
plt.ylabel("Predicted RUL", fontsize=14)

plt.legend()
plt.grid(alpha=0.3)

plt.show()
```



Feature Importance Graph

In [37]:
```
feature_importance = pd.DataFrame({
    "Feature": X.columns,
    "Importance": model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by="Importance",
    ascending=False
)

colors = ["green", "limegreen", "gold", "orange", "red"]

plt.figure(figsize=(8,5))

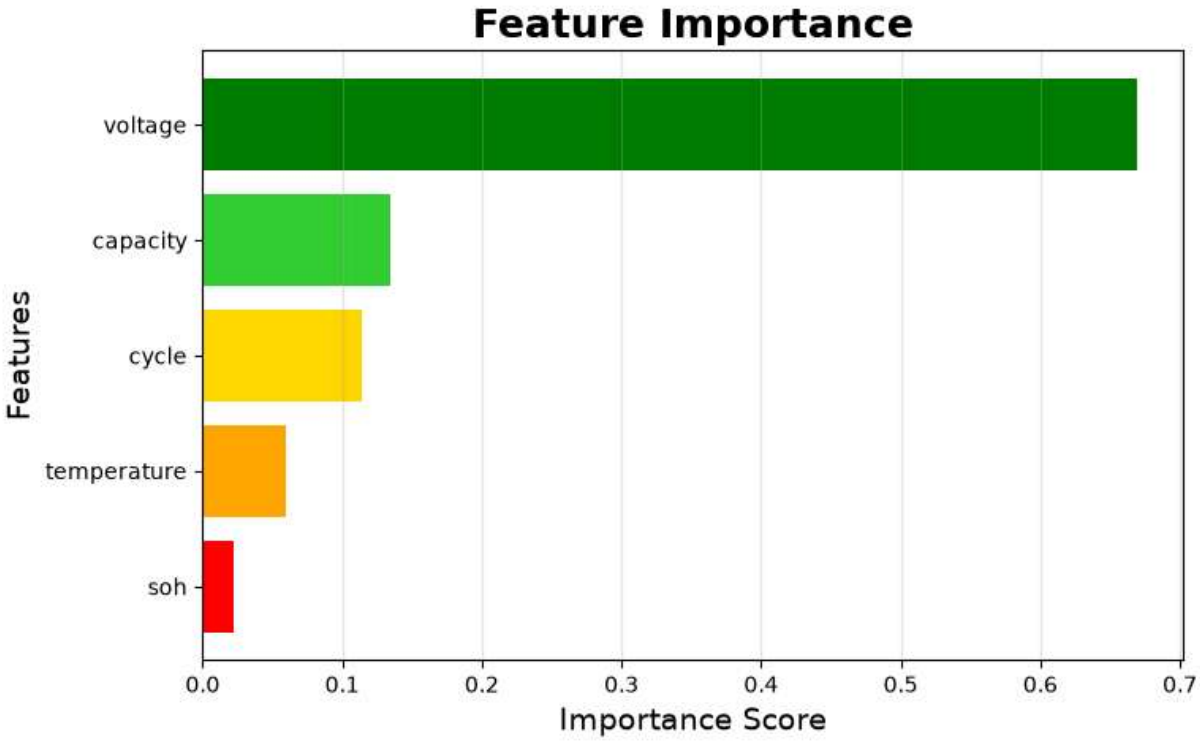
plt.barh(
    feature_importance["Feature"],
    feature_importance["Importance"],
    color=colors
)

plt.title("Feature Importance", fontsize=18, fontweight="bold")
plt.xlabel("Importance Score", fontsize=14)
plt.ylabel("Features", fontsize=14)
```

```
plt.grid(axis="x", alpha=0.3)

plt.gca().invert_yaxis()

plt.show()
```



Prediction Results

In [38]:
```
results = pd.DataFrame({
    "Actual RUL": y_test.values,
    "Predicted RUL": y_pred
})

results.head(10)
```

Out[38]:

	Actual RUL	Predicted RUL
0	7	7.65
1	37	37.78
2	151	147.61
3	130	131.55
4	119	119.37
5	24	26.73
6	90	88.45
7	100	97.65
8	18	19.15
9	116	117.27

Prediction Using New Battery Data

In [39]:
```
new_battery = pd.DataFrame({
    "cycle": [500],
    "voltage": [3.85],
    "temperature": [30],
    "capacity": [1.75],
    "soh": [88]
})

prediction = model.predict(new_battery)

print("Prediction Remaining Useful Life (RUL):", prediction[0])
```

```
Prediction Remaining Useful Life (RUL): 9.32
```

Conclusion

This project successfully predict the Remaining Useful Life (RUL) of lithium-ion batteries using machine learning. The Random Forest Regressor model achieved high prediction accuracy with an R^2 score of 98.89%. The system can help monitor battery health, improve maintenance planning, and reduce battery failure risks in real-world applications.

Project Completed

Thank You!

In [40]:
```
import joblib
```

```
joblib.dump(model, "../Models/model.pkl")

print("Model Saved Successfully!")
```

```
Model Saved Successfully!
```

In [ ]: